Pradeep Raj Katipagala

24145399

GitHub link: https://github.com/pradeep271/Support-Vector-Machine/tree/main

SUPPORT VECTOR MACHINES WITH HYPER PARAMETER TUNING

Before getting into the topic of SVM let us first recall about Machine Learning and its types, how they differ and their use cases.

1. Introduction to Machine learning

Machine Learning is a branch of Artificial Intelligence that allows systems or engines to learn useful patterns from data in making impactful decisions without explicitly programmed. ML models do not rely on predefined rules they improve their performance as they get exposed to more data and insights.

	Type	Usage	Examples
1	Supervised Learning	Target Variable known	SVM, KNN, Regression…
2	Unsupervised Learning	Target Variable unknown	K-means, DBSCAN…
3	Semi-supervised Learning	Categorical Target Variable	Text classification…
4	Reinforcement Learning	Goal based	Q, R, TD learning

Table 1: Types of Machine Learning

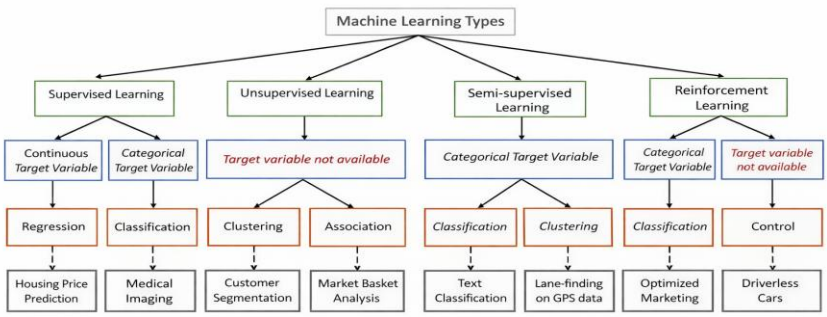

Figure 1: Machine Learning use cases

From the above we can conclude that SVM is a Supervised Learning model which uses labeled data for prediction and comes under classification method. Now let us go deep into SVM the Support Vector Machines its usage Kernels and Hyper tuning.

2. Support Vector Machines (SVM)

SVM is one of the most powerful supervised learning algorithms used for both classification and regression which are particularly effective in high-dimensional spaces. The main aim of SVM is to find a hyperplane that is best fit for separating the classes. The optimal hyperplane is the one which maximizes the margin, that means the distance between the boundary of the and the closest data points.

- In 2D -> Line
- In 3D-> Plane
- In higher dimensions-> a hyperplane

Types of SVM:

- Linear-> Works when data can be separated by straight line.
- Non-Linear->Transform data into higher dimensions using kernels.

Support Vectors are data points which are closest to the decision boundary, these points determine the position and orientation of the boundary and when the points move, the boundary changes which make it critical. SVM usually uses Margin Maximization for better generalization.

$$\text{Margin} = 2/||w||$$

- w = weight vector of hyperplane.

Kernal:

The kernel allows SVM to work in higher dimensional space without actual computing of explicit transformations, instead of mapping the data manually, kernel is used to compute similarity between data points.

	Kernel	**Use Case**
1	Linear	Simple, Linear separable data
2	Polynomial	Data having curved relations
3	RBF(Gaussian)	Complex, nonlinear decision boundaries
4	Sigmoid	Like Neural networks

Table 2: Common Kernels in SVM

Applications of SVM in real life:

- Face Detection
- Bioinformatics
- Text and hypertext classification
- Classification of images

3. Dataset

The dataset is generated using make_classification which is a function from sklearn.datasets commonly used for generating synthetic classification datasets for training and testing Machine Learning models.

Parameter	Value	Meaning
n_samples	300	Generate 300 data points
n_features	2	Each data point has 2 features
n_redundant	0	No duplicate or dependent features
n_informative	2	No noise only features both features are useful
n_clusters_per_class	1	Each class forms 1 cluster
class_sep	1.5	Controls how far apart classes are from each other
random_state	42	Ensure reproducibility

Table 3: Parameters in dataset

The output of the dataset generation code gives us a 2D linearly separable classification dataset with controlled complexity and it is ideal for visualizing and evaluating machine learning models like SVM.

X-> Feature matrix (shape: 300*2)

y-> target labels

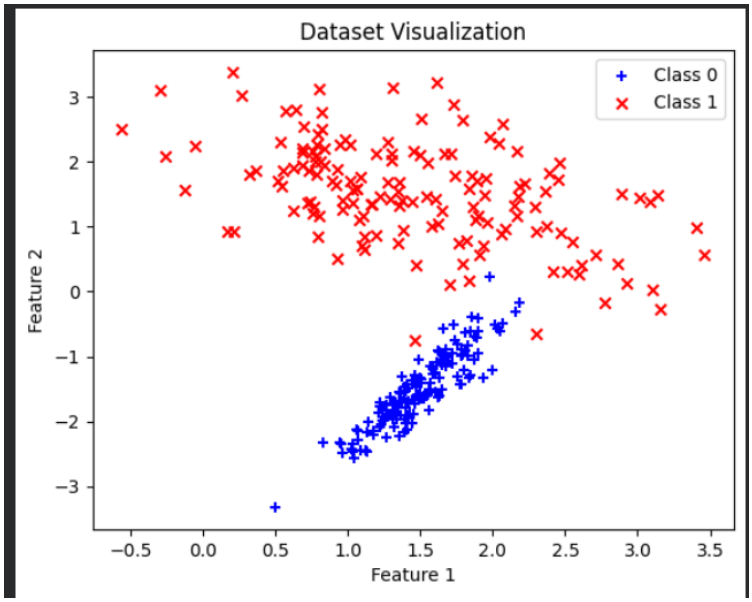

Figure 2. Dataset Visualization

For colorblind friendly the data points are set to shapes like ‘x’ and ‘+’ where it can be easily identified and even the projections are mentioned as Class 0 and Class 1. This step makes a unique way for identification and separating both classes and get useful insights from the visualization. The X axis contains the Feature 1 and Y axis contains Feature 2 distributions.

4. Decision Boundary and Kernel Comparison

A key idea in classification problems is a decision boundary, which is the surface in the feature space that divides several classes. The decision boundary in the context of the created dataset (which has two features) is a line or curve in a two-dimensional plane that separates the data points into their corresponding classes (0 and 1). The dataset is a great tool for learning model behavior because it is made using make_classification with n_features=2, which makes it easy to visualize the border.

A subset of training points known as support vectors determines the decision boundary in Support Vector Machines (SVM). These are the points nearest to the boundary, and they are essential in determining its orientation and location. To improve generalization to new data, SVM aims to maximize the margin between the classes. The choice of kernel function has a significant impact on the decision boundary's complexity and shape. SVM can handle both linear and non-linear classification problems by using kernels to convert the data into a higher-dimensional space.

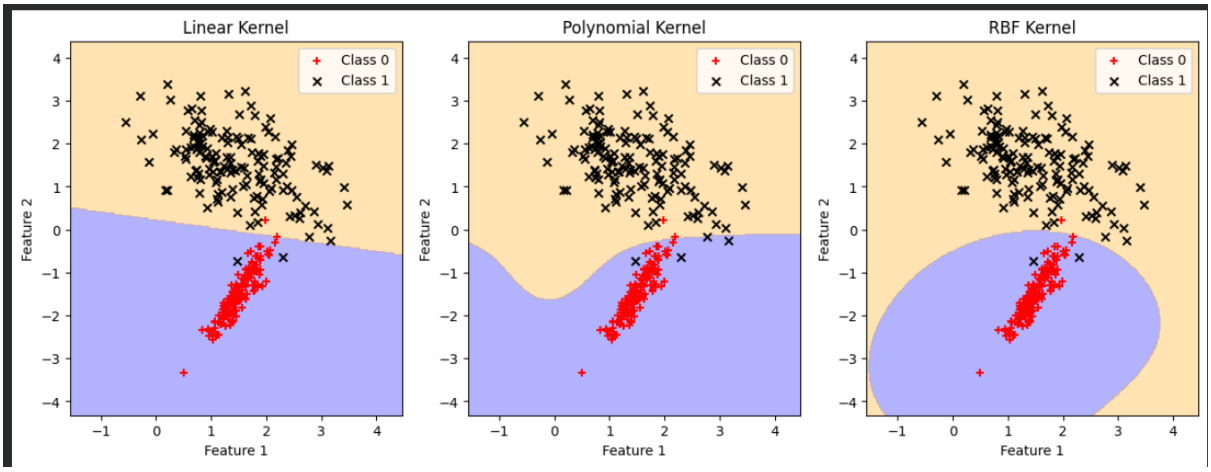

Figure 3. Kernal Comparison and decision boundaries

Kernel Linear: A straight-line decision boundary is produced by the linear kernel. When the data is roughly or linearly separable, it works well. Given the dataset, a linear kernel might function rather well because class_sep=1.5 offers moderate separation. However, if the data distribution has a small amount of overlap or curvature, it may have trouble.

Kernel Polynomial: By permitting curved decision boundaries, the polynomial kernel introduces non-linearity. By changing the polynomial's degree, it can simulate more intricate interactions. Although this kernel can better fit data with curved patterns, if the degree is too high, it may cause overfitting, particularly with relatively small datasets like the one employed here.

Kernel Gaussian RBF: The most popular kernel, which is also utilized in your code, is RBF. By taking the distance between data points into account, it generates extremely flexible, non-linear decision limits. The degree to which the boundary matches the data is determined by the parameter gamma. A large gamma creates a more complicated boundary that could overfit the training set, whereas a lower gamma yields a smooth boundary. The RBF kernel works very well with this dataset because it can adjust to minute patterns and changes in the data.

5. Hyperparamter Tuning

The process of choosing the ideal collection of parameters to regulate a machine learning model's behavior is known as hyperparameter tuning. Hyperparameters are predetermined and have a major impact on model performance, in contrast to model parameters, which are learned during training. This approach uses GridSearchCV on a Support Vector Machine (SVM) with an RBF kernel to tune hyperparameters.

The search space is specified by the parameter grid:

C: Manages the trade-off between reducing classification error and increasing margin. Larger values strive for flawless classification (risk of overfitting), whereas smaller values permit greater misclassification (better generalization).

Gamma: Determines the extent to which a single data point has an impact. A large gamma produces a more complicated border, whereas a lower gamma produces a smoother decision boundary.

Kernel: Since it works well with non-linear data, it was fixed as "rbf."

GridSearchCV uses 5-fold cross-validation (cv=5) to do a thorough search over all possible combinations of these parameters. This reduces overfitting and increases reliability by ensuring that the model is assessed on several subsets of the data. The combination that produces the best performance is provided by the output, grid.best_params_. This methodical technique improves forecast accuracy and guarantees the best model selection.

6. Model Evaluation and Report

A crucial stage in any machine learning process is model evaluation, which assesses a trained model's performance on untested data. Both quantitative metrics and visual comparison methods are used in this implementation's evaluation process. The objective is to evaluate how well various Support Vector Machine (SVM) configurations and kernel functions perform on the produced dataset.

The classification report generated from the best SVM model provides a detailed evaluation of its performance on the test dataset. It includes key metrics such as precision, recall, F1-score, and support for each class, along with overall accuracy.

```
best_model = grid.best_estimator_
y_pred = best_model.predict(X_test)

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      0.98      0.99        46
           1       0.98      1.00      0.99        44

    accuracy                           0.99        90
   macro avg       0.99      0.99      0.99        90
weighted avg       0.99      0.99      0.99        90
```

Figure 4. Classification Report

Performance by Class

Regarding Class 0:

Precision = 1.00=> There are no false positives because every projected instance of class 0 is accurate.

Recall = 0.98 => There are very few false negatives because 98% of real class 0 cases are successfully detected.

F1-score = 0.99 => A good ratio of recall to precision.

Support = 46 => There are 46 real class 0 samples.

Regarding Class 1:

Precision = 0.98 => 98% of class 1 predictions are accurate.

Recall = 1.00 => Every real class 1 sample has an accurate classification (no false negatives).

F1-score = 0.99 => Outstanding recall and precision balance.

Support = 44 => There are 44 class 1 samples.

The model performs remarkably well, with very high precision and recall for both classes, according to the results. The near-perfect accuracy indicates that the underlying patterns in the dataset have been successfully captured by SVM with adjusted hyperparameters. Furthermore, the model's balanced performance in both classes suggests that it is not biased in favor of any one class. Overall, the classification report verifies that the chosen model is quite dependable and does a good job of generalizing to new data, which makes it appropriate for classification tasks on comparable datasets.

7. Accuracy Comparison Across Kernels

Using classification accuracy on test dataset, the effectiveness of various Support Vector Machine (SVM) kernels—linear, polynomial, and radial basis function (RBF)—was assessed. According to the findings, the polynomial kernel attained a somewhat lower accuracy of 0.978, whilst the linear and RBF kernels both attained an accuracy of 0.989.

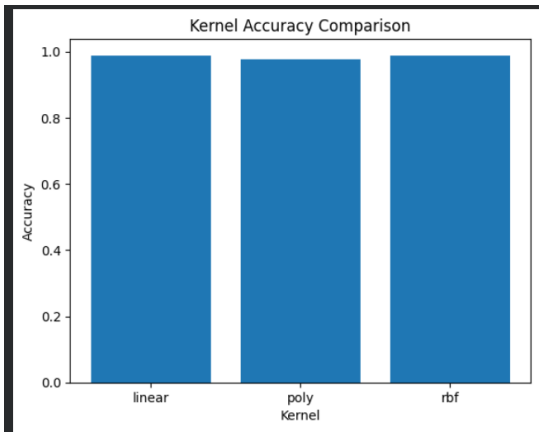

Figure 5. Accuracy Comparison

Because the dataset is approximately linearly separable and moderately well-separated (class_sep=1.5), the linear kernel performs well. As a result, the linear model can efficiently and with little error establish a straight decision boundary. High precision is also attained by the RBF kernel, which is renowned for its adaptability. In this instance, its performance is comparable to that of the linear kernel, suggesting that the dataset can be accurately classified without the need for extremely complicated changes.

liner: 0.989

polynomial: 0.978

rbf: 0.989

Overall, the findings imply that when the data is well-structured, simpler models (linear kernel) can function just as well as more complicated ones (RBF). This emphasizes how crucial it is to choose a suitable kernel based on the features of the dataset rather than using more complicated models by default.

8. Hyperparameter Accuracy Heatmap

Using the RBF kernel, the heatmap shows how the SVM model's performance changes with various combinations of the hyperparameters C and gamma. The heatmap is a useful visual aid for hyperparameter tuning since each cell shows the classification accuracy attained for a particular pair of values.

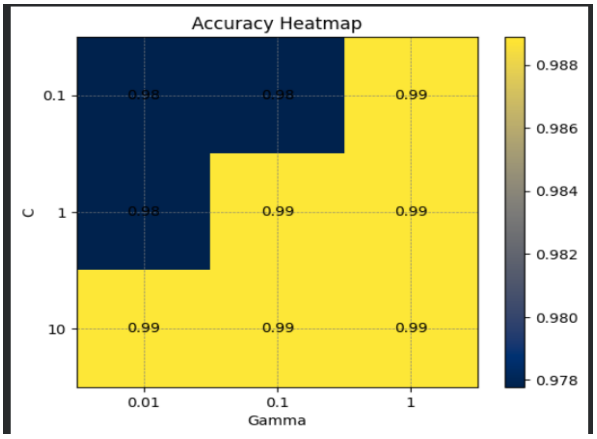

Figure 6. Accuracy Heatmap

It is clear from the results that the model performs well overall, routinely achieving high accuracy values between 0.98 and 0.99. The accuracy is somewhat lower (0.98) with lower values of C (0.1) and gamma (0.01, 0.1), indicating underfitting because of a smoother decision boundary. The accuracy increases to 0.99 as C and gamma values rise, indicating a better match to the data.

Because performance does not significantly change across parameter combinations, the heatmap further demonstrates the model's relative stability. This shows how well the SVM model performs on this dataset. Overall, the model successfully balances bias and variation in the ideal region, which is found at larger values of C and gamma.

*For colorblind friendly the heatmap has grid lines and points are within the lines.

9. Conclusion

This study shows how Support Vector Machines (SVM) use an organized method to handle categorization challenges. Various kernels and hyperparameters were examined using a synthetic dataset. The results demonstrate that the data is well-structured, with linear and RBF kernels achieving the maximum accuracy (0.989). Because of parameter sensitivity, the polynomial kernel performed marginally worse. By tweaking C and gamma, GridSearchCV hyperparameter tuning enhanced performance. Strong generalization capacity was confirmed by the final model's good precision, recall, and F1-scores (~0.99).

10. References

Cortes, C., & Vapnik, V. (1995). *Support-vector networks*. Machine Learning.

Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. JMLR.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.